The child classes automatically have `$quoteStash` and `getQuote()` members, but the children re-declare `$quoteStash` with their specialized data. This is another feature of inheritance: child classes can replace the parent implementation with their own; this is known as `overriding`. Note the use of the `protected` keyword here – `protected` members, like `private`, are not accessible outside the class but are made available for use or overloading in the child class. `private` members are not inherited by children.

If we instantiate one of the child classes from above, we can see that the inherited method is available for use:

```
$lebowski = new LebowskiBot();
print $lebowski->getQuote();
```

```
Output:
That's just, like, your opinion, man.
```

Parent and child classes also have a special relationship in the type declaration system. Any type declaration that is a class will also accept an instance of a subclass. For example:

```
function printQuote(QuoteBot $bot): void
{
    print $bot->getQuote();
}
```

We declare a simple function to make use of an instance of `QuoteBot`, passed in as its only argument. It's perfectly fine to pass in a child of `QuoteBot`:

```
$fink = new FinkBot();
printQuote($fink);
```

```
Output:
I'll show you the life of the mind!
```

As stated, a child class can override a parent member, but it cannot reduce visibility; therefore, all of the public methods and properties of a parent will be available for a child class too. This restriction on inheritance means that substituting a child for a parent will always be safe, another important OOP principle.